Хешування

Хеш-функція, або геш-функція— функція, що перетворює вхідні дані будь-якого (як правило великого) розміру в дані фіксованого розміру.

Хешування (гешування, англ. hashing) — перетворення вхідного масиву даних довільної довжини у вихідний бітовий рядок фіксованої довжини. Такі перетворення також називаються хеш-функціями, або функціями згортання, а їхні результати називають хешем, хеш-кодом, хеш-сумою, або дайджестом повідомлення (англ. message digest).

Хеш-функція використовується зокрема в структурах даних — хеш-таблицях, широко вживаних у програмному забезпеченні для швидкого пошуку даних. Хеш-функції використовуються для оптимізації таблиць та баз даних за рахунок того, що у однакових записів однакові значення хеш-функції. Такий підхід пошуку дублікатів ефективний в файлах великого розміру. Прикладом цього буде знаходження подібних ділянок у послідовностях ДНК. Криптографічна геш-функція дозволяє легко перевірити, що деякі вхідні дані зіставляються із заданим значенням хешу, але, якщо вхідні дані невідомі, то навмисно важко відновити вхідне значення (або еквівалентну альтернативу), знаючи збережене значення хеш-функції. Це використовується для забезпечення цілісності переданих даних, і є будівельним блоком для HMACs, які забезпечують аутентифікацію повідомлень.

Хешування застосовується для:

- побудови асоціативних масивів, пошуку дублікатів в серіях наборів даних,
- побудови унікальних ідентифікаторів для наборів даних,
- контрольного підсумовування з метою виявлення випадкових або навмисних помилок при зберіганні або передачі,
- зберігання паролів в системах захисту (у цьому випадку доступ до області пам'яті, де знаходяться паролі, не дозволяє відновити сам пароль),
- вироблення електронного підпису (на практиці часто підписується не саме повідомлення, а його еш-образ).

У загальному випадку однозначної відповідності між вихідними даними і хеш-кодом немає в силу того, що кількість значень хеш-функцій менша, ніж число варіантів значень вхідного масиву. Існує безліч масивів з різним вмістом, що дають однакові хеш-коди — так звані колізії. Імовірність виникнення колізій відіграє важливу роль в оцінці якості хеш-функцій.

Розроблено багато алгоритмів хешування з різними властивостями (розрядність, обчислювальна складність, криптостійкість тощо). Вибір тієї чи іншої хеш-

функції визначається специфікою розв'язуваної задачі. Найпростішими прикладами геш-функцій можуть служити контрольна сума або CRC.

Хеш-функції пов'язані (і їх часто плутають) з контрольною сумою, контрольними цифрами, відбитками пальців, рандомізацією функцій, кодами, що виправляють помилки, і з шифрами. Хоча ці поняття певною мірою збігаються, кожен з них має свою власну область застосування і вимоги і є розробленим і оптимізованим по-різному.

Хеш-таблиця — структура даних, що реалізує інтерфейс асоціативного масиву, а саме, вона дозволяє зберігати пари (ключ, значення) і здійснювати три операції: операцію додавання нової пари, операцію пошуку і операцію видалення за ключем.

Існує два основних варіанта хеш-таблиць:

- з ланцюжками
- з відкритою адресацією.

Виконання операцій в хеш-таблиці починається з обчислення хеш-функції від ключа. Отримане хеш-значення i = hash(key) відіграє роль індексу в масиві H. Після цього операція (додавання, видалення, пошук) перенаправляється об'єктові, який зберігається у відповідній комірці масиву H[i].

Ситуація, коли для різних ключів отримується одне й те саме хеш-значення, називається **колізією**. Такі події непоодинокі — наприклад, при додаванні в геш-таблицю розміром 365 комірок усього лише 23-х елементів ймовірність колізії вже перевищує 50 відсотків (якщо кожний елемент може з однаковою ймовірністю потрапити до будь-якої комірки). Через це механізм розв'язання колізій — важлива складова будь-якої хеш-таблиці.

В деяких особливих випадках вдається взагалі уникнути колізій. Наприклад, якщо всі ключі елементів відомі заздалегідь (або дуже рідко змінюються), тоді для них можна знайти деяку досконалу хеш-функцію, яка розподілить їх за комірками хеш-таблиці без колізій. Хеш-таблиці, які використовують подібні Хеш-функції, не потребують механізму розв'язання колізій, і називаються Хеш-таблицями з прямою адресацією.

Властивості Хеш-таблиці

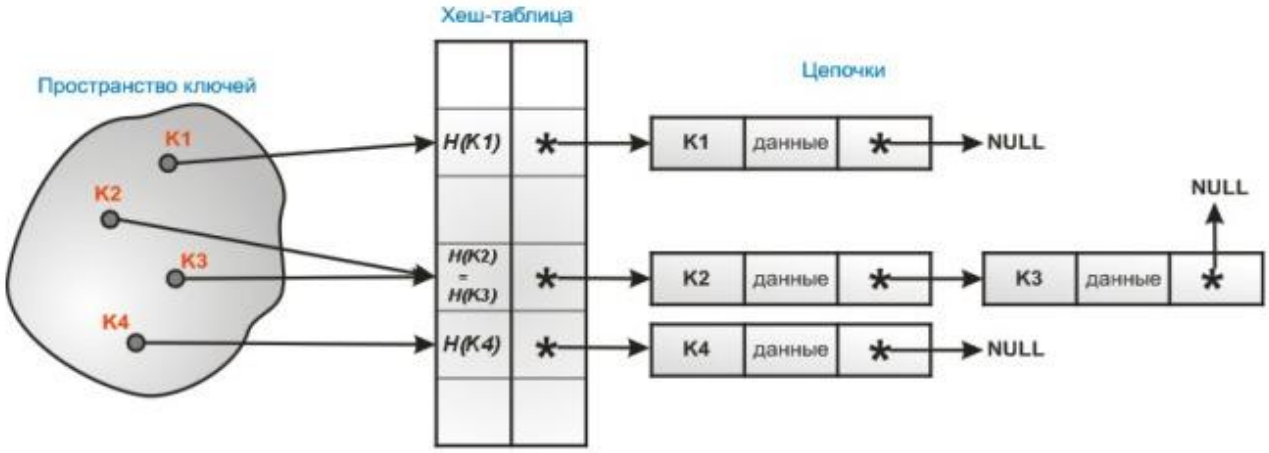
- при деяких розумних припущеннях, всі три операції (пошук, вставлення і видалення елементів) зазвичай виконується за час O(1). Але при цьому не гарантується, що час виконання окремої операції малий, з певною імовірністю час може бути доданий до часу пошуку у список;.
- з ростом коефіцієнта заповнення таблиці ця імовірність, і, відповідно, середній час виконання операцій, ростуть. Тому при досягненні деякого

значення коефіцієнта заповнення необхідно здійснювати перебудову індексу хеш-таблиці: збільшити розміри масиву H і заново додати в порожню геш-таблицю всі пари.

Розв'язання колізій

1. Метод ланцюжків
2. Відкрита адресація
3. Об'єднане хешування
4. Хешування зозулею
5. Hopscotch гешування

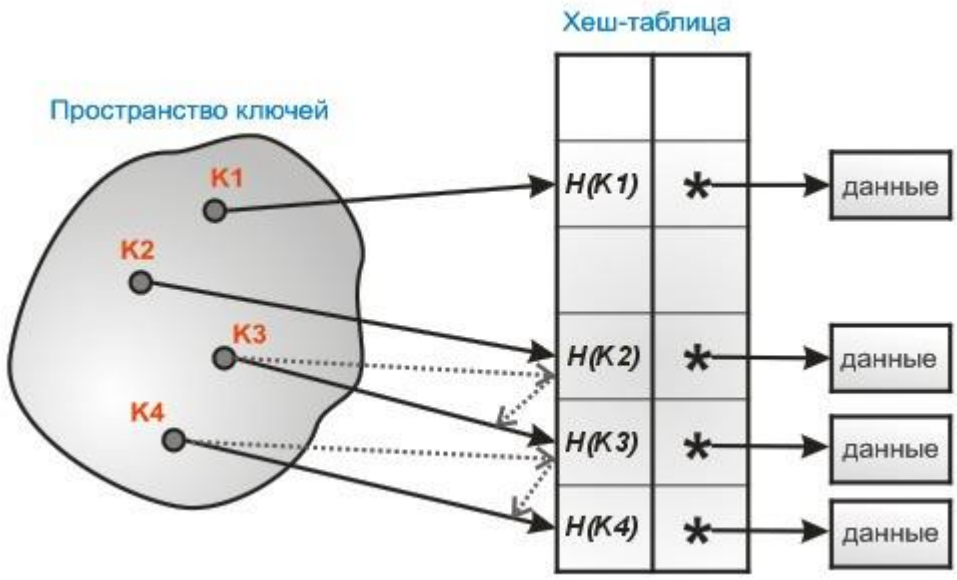
Розв'язання колізій за допомогою ланцюжків.



Кожна комірка масиву H є вказівником на зв'язаний список (ланцюжок) пар ключ-значення, відповідних одному і тому самому хеш-значенню ключа. Колізії просто призводять до того, що з'являються ланцюжки довжиною більше одного елемента.

Операції пошуку або видалення елемента вимагають перегляду всіх елементів відповідного ланцюжка, щоб знайти в ньому елемент з заданим ключем. Для додавання нового елемента необхідно додати елемент в кінець або початок відповідного списку, і, у випадку якщо коефіцієнт заповнення стане занадто великим, збільшити розмір масиву H і перебудувати таблицю.

При припущенні, що кожний елемент може потрапити в будь-яку позицію таблиці H з однаковою ймовірністю і незалежно від того, куди потрапив будь-який елемент, пересічний час роботи операції пошуку елемента складає $\Theta(1 + \alpha)$, де α — коефіцієнт заповнення таблиці.Такий метод ще називають **відкритим хешуванням.**



В методі, що має назву відкритої адресації, всі записи зберігаються в самому масиві H. Коли додається новий запис, масив перевіряється в певному порядку, доки не буде знайдена вільна комірка, куди буде доданий елемент. У випадку пошуку елемента, масив сканується в тій самій послідовності, доки потрібний запис або порожня комірка не буде знайдена. Останнє — означає відсутність елемента. Назва «відкрита адресація» показує, що положення («адреса») елемента не визначається його хеш-значенням. Цей метод також називають **закритим хешуванням.**

Загалом, послідовність, у якій переглядаються комірки хеш-таблиці, залежить лише від ключа елемента. Тобто це послідовність h0(x), h1(x), …, hn-1(x), де x — ключ елемента, а hi(x) — довільна функція, яка порівнює кожен ключ комірки з хеш-таблицею. Перший елемент послідовності, зазвичай, дорівнює значенню деякої хеш-функції від ключа, а наступні обчислюються одним з наведених нижче способів. Для успішної роботи алгоритму пошуку, послідовність перебору має бути такою, щоб всі комірки гхш-таблиці виявились переглянутими рівно по одному разу.

Алгоритм пошуку переглядає всі комірки хеш-таблиці в тому самому порядку, що і при вставці, доки не знайдеться або елемент з шуканим ключем, або вільна комірка (що означає відсутність елемента в хеш-таблиці).

Видалення елементів у такій схемі складніше. Зазвичай воно здійснюється таким чином: заводять булевий прапорець для кожної комірки. Він має означати видалений цей елемент з таблиці чи ні. Тобто видалення елемента означає встановлення цього прапорця. При цьому доведеться модифікувати процедуру пошуку існуючого елемента так, щоб вона вважала видалені комірки зайнятими,

а в процедурі додавання — вона вважала їх вільними і скидала прапорець при додаванні елемента.

Варіанти переборів (нумерація елементів послідовності спроб і комірок хеш-таблиці при переборі ведеться від нуля, а N — розмір еш-таблиці.

- Лінійне зондування: комірки хеш-таблиці послідовно переглядаються з деяким фіксованим інтервалом k між комірками (зазвичай,), тобто і-й елемент послідовності — це комірка з номером (hash(x) + ik) mod N. Для того, щоб всі комірки виявились перебраними по одному разу, необхідно, щоб k було взаємно-простим з розміром геш-таблиці.
- Квадратичне зондування: інтервал між комірками з кожним кроком збільшується на константу, що задається в загальному вигляді з допомогою деякого квадратичного поліному k1i+k2i2. У найпростішому випадку k1 = 0, k2 = 1 для i = 0,1,2,3,… отримаємо:
  h0 = hash(x) mod N, h1 = (hash(x) + 12) mod N, h2 = (hash(x) + 22) mod N, h3 = (hash(x) + 32) mod N, …
- Подвійне хешування: інтервал між комірками фіксований, як при лінійному переборі, але, на відміну від нього, розмір інтервалу обчислюється другою, допоміжною хеш-функцією, тобто може бути різним для різним ключів. Значення цієї хеш-функції мають бути ненульовими і взаємно простими з розміром хеш-таблиці, що найпростіше всього зробити, якщо взяти просте число як розмір, і вимагати, щоб допоміжна хеш-функція приймала значення від 1 до N − 1.

Недоліком усіх схем відкритої адресації є те, що кількість елементів, які можуть зберігатися в таблиці може досягти кількість комірок в масиві.

Схеми відкритої адресації також накладають суворіші вимоги до хеш-функції: окрім рівномірного розподілу значень по всьому масиву, функція має також мінімізувати скупчення хеш-значень, що слідують одна за одною в послідовності спроб.

Відкрита адресація зберігає пам'ять тільки у випадку якщо елементи малого розміру і коефіцієнт завантаження не дуже малий. Якщо коефіцієнт завантаження близький до нуля, відкрита адресація марнотратна навіть якщо кожен елемент займає лише два машинних слова.

Відкрита адресація уникає витрат на розміщення кожного нового елемента, і може бути реалізована навіть у відсутності розподільника пам'яті. Вона також уникає додаткових посилань доступу до набору елементів з певним однаковим значенням хеш-функції. З малими розмірами елементів, цей фактор може додати продуктивності порівняно з таблицею організованою методом ланцюжків особливо при пошуку.

Зазвичай відкрита адресація — поганий вибір для великих елементів, через те, що такі елементи заповнюють весь кеш процесора (нівелюючи переваги кеша), також значна кількість місця втрачається через велику кількість порожніх комірок. Якщо відкрита адресація зберігає тільки вказівники на елементи, вона використовує кількість пам'яті, яку можна порівняти з використанням пам'яті методом ланцюжків, але втрачає перевагу в швидкості.

Відкриту адресацію краще використовувати для хеш-таблиць з малими елементами, які зберігаються в таблиці. Вони особливо підходять для елементів в одне слово або менших.

У випадку коли очікується високий коефіцієнт завантаження, елементи великі за розміром, або можуть мати різні розміри, метод ланцюжків показує таку ж або кращу продуктивність.

Об'єднане хешування

Гібрид методу ланцюжків та відкритої адресації (англ. Coalesced hashing), що розв'язує колізії з допомогою додаткових ланок між вузлами зв'язаних списків всередині таблиці. Як і метод ланцюжків, не має кластерних ефектів і хеш-таблиця може бути ефективно заповнена.

Хешування зозулею

Ще одна альтернатива відкритої адресації — англ. Cuckoo hashing,— забезпечує постійний час пошуку і сталий амортизований час для вставок і вилучень. Цей метод використовує дві чи більше хеш-функції, а це означає, що будь-яка пара ключ/значення може знаходитись в двох або більше місцях. Для пошуку використовується перша хеш-функція; якщо ключ/значення не знайдено, то використовується друга хеш-функція, і так далі. Якщо колізія відбувається під час вставки, то ключ повторно хешується другою хеш-функцією, щоб відобразити його в інший бакет. Якщо всі функції хешування використані, а колізія не розв'язана, то старий ключ на місці колізії видаляється, щоб звільнити місце для нового ключа, і цей попередній старий ключ повторно хешується однією з наступних хеш-функцій, які відображають його в інший бакет. Якщо це також призводить до колізії, то процес повторюється, поки колізія не зникне або процес пройде через всі бакети в таблиці. Таким шляхом оптимізується використання пам'яті.

Hopscotch гешування

Інший альтернативний метод, поєднує в собі хешування зозулею та лінійне з але таким чином, щоб обійти обмеження цих методів. Зокрема, він добре працює навіть тоді, коли коефіцієнт завантаження таблиці зростає до 0,9.

У хеш-таблиці замість безпосереднього використання ключа як індексу масиву, індекс обчислюється за значенням ключа. Функція, що відображає елемент множини ключів {0, 1, ..., n-1} на множину індексів {0, 1, ..., m-1} (m <n), називається хеш-функцією. Якщо два ключі хешуються в одну й ту саму комірку, то говорять про виникнення колізії. За способом вирішення колізій розрізняють:

- відкрите хешування — усі елементи, що хешуються в одну комірку, об’єднуються у зв’язний список
- закрите хешування — усі елементи зберігаються безпосередньо у хеш-таблиці, при потраплянні у зайняту комірку обирається послідовність інших хеш-значень.

Колізією хеш-функції H називаються два різних вхідних блоки даних x і y таких, що H(x)=H(y).

Колізії існують для більшості хеш-функцій, але для «вдалих» хеш-функцій частота їх виникнення близька до теоретичного мінімуму. В деяких окремих випадках, коли множина різних вхідних даних є скінченною, можна задати ін'єктивну хеш-функцію, за визначенням без колізій. Однак, для хеш-функцій, які приймають вхідні дані змінної довжини і повертають хеш постійної довжини, колізії обов'язково будуть існувати, оскільки хоча б для одного значення хеш-функції відповідна йому вхідна множина значень буде нескінченною — і будь-які два значення з цієї множини утворять колізію.

Одним з найбільш простих і універсальних методів пошуку колізій є атака «днів народження». За допомогою цієї атаки знаходження колізії для хеш-функції розрядності n біт потрібно в середньому близько $2^{n/2}$ операцій. Через це n-бітова хеш-функція вважається криптостійкою, якщо обчислювальна складність знаходження колізій для неї близька до $2^{n/2}$.

Так званий "парадокс дня народження" полягає в наступному.

- Припустимо, кількість вихідних значень хеш-функції H одно n.
- Яким має бути число k, щоб для конкретного значення X і значень $Y_1 \ldots Y_k$ ймовірність того, що хоча б для одного Y_i виконувалося рівність H (X) = H (Y) була б більше 0,5.
- Для одного Y ймовірність того, що H (X) = H (Y), дорівнює 1 / n.
- Відповідно, ймовірність того, що H (X) \ ne H (Y), дорівнює 1 - 1 / n.
- Якщо створити k значень, то ймовірність того, що ні для одного з них не буде збігів, дорівнює добутку ймовірностей, що відповідають одному значенню, тобто (1 - 1 / n) k.
- Отже, ймовірність, принаймні, одного збігу дорівнює1 - (1 - 1 / n) k

- Зробити обрахунки за формулою бінома Н’ютона.

Таким чином можна з'ясувати, що для m-бітового хеш-коду досить вибрати 2m-1 повідомлень, щоб ймовірність збігу хеш-кодів була більше 0,5.

Приклад

Припустимо, що використовується 64-бітний хеш-код. Можна вважати, що це цілком достатня і, отже, безпечна довжина для хеш-коду. Наприклад, якщо зашифрований хеш-код С передається з відповідним незашифрованим повідомленням М, то для взлома необхідно буде знайти М 'таке, що

Н (М ') = Н (М)

для того, щоб підмінити повідомлення, що надходило до одержувача. В середньому потрібно перебрати 263 повідомлень для того, щоб знайти таке, у якого хеш-код дорівнює перехопленому повідомленням.

Проте, можливі різного роду атаки, засновані на "парадоксі дня народження". Можлива наступна стратегія перехоплення;

Створюється 2m / 2 варіантів повідомлення, кожне з яких має певний сенс. Потім готується така ж кількість повідомлень, кожне з яких є підробленими і призначеним для заміни цього повідомлення.

Два набору повідомлень порівнюються в пошуках пари повідомлень, що мають однаковий хеш-код. Імовірність успіху відповідно до "парадоксом дня народження" більше, ніж 0,5. Якщо відповідна пара не знайдена, то створюються додаткові вихідні та підроблені повідомлення до тих пір, поки не буде знайдена пара.

Атакуючий пропонує відправнику вихідний варіант повідомлення для підпису. Цей підпис може бути потім приєднана до підробленим варіанту для передачі одержувачу. Так як обидва варіанти мають один і той же хеш-код, буде створений однаковий підпис. Противник буде впевнений в успіху, навіть не знаючи ключа шифрування.

Таким чином, якщо використовується 64-бітний хеш-код, то необхідна складність обчислень становить близько 232.

.